

Relatório Técnico - AutoTarget

Arquitetura Concorrente, Reconciliação de Dados e Agentes Autônomos
(Avaliação 2)

Alexandre Reis Francisco Júnior

26 de maio de 2026

Resumo

Este relatório descreve detalhadamente as escolhas arquiteturais e a implementação técnica da Avaliação 2 do projeto AutoTarget. O sistema evoluiu de uma prova de conceito inicial para um ecossistema complexo e concorrente de simulação tática. Foram aplicados conceitos avançados de Sistemas Operacionais (Threads, Locks, Thread Affinity), Processamento de Sinais (Sensores com ruído estocástico, Reconciliação de Dados via Álgebra Linear) e Inteligência Artificial (Agentes Racionais baseados em Função de Utilidade). O documento destrincha o modelo de sincronização empregado para evitar condições de corrida, a estruturação matemática do filtro de sensores e a heurística de tomada de decisão autônoma.

Conteúdo

1	Introdução	4
2	Arquitetura Concorrente e Sincronização	4
2.1	Modelo Multithreaded	4
2.2	Prevenção de Condições de Corrida e Sincronização Granular	4
3	API de Afinidade (Thread Affinity) via Reflection	6
4	Modelagem Estocástica e Reconciliação de Dados	6
4.1	Sensores Virtuais Ruidosos	6
4.2	Reconciliação Matemática de Medições	7
5	Modelagem Comportamental: Agentes Racionais e Autonomia	8
5.1	Cinemática Horizontal e Prevenção de Sobreposição	8
5.2	Função de Utilidade e Alocação Dinâmica	9
6	Pipeline de Renderização Não-Bloqueante	10
7	Conclusão	10

1 Introdução

O projeto **AutoTarget** consiste em um simulador de combate bidimensional onde duas frentes autônomas (Sistema A e Sistema B) competem em tempo real pela neutralização de alvos móveis. A complexidade do sistema reside na natureza estocástica do ambiente e na necessidade de gerenciar múltiplos recursos computacionais simultaneamente, sem degradação de performance.

O objetivo desta etapa do projeto foi a transição para um modelo puramente multithreaded e a introdução de incertezas sistêmicas (ruídos de sensoriamento), exigindo a implementação de algoritmos de otimização e controle para a operação estável da simulação.

2 Arquitetura Concorrente e Sincronização

A base do AutoTarget é fortemente orientada a eventos concorrentes. Diferente de aplicações sequenciais tradicionais, o simulador delega a responsabilidade de processamento para dezenas de threads simultâneas, necessitando de uma rigorosa orquestração de memória compartilhada.

2.1 Modelo Multithreaded

O fluxo principal, `Jogo.java`, orquestra o ciclo de vida da partida. No entanto, cada entidade física na tela — `Canhao`, `Alvo` e `Projetil` — é encapsulada em sua própria `Thread`. Isso significa que as atualizações de cinemática (cálculo de deslocamento e trajetória) ocorrem de forma paralela e independente.

2.2 Prevenção de Condições de Corrida e Sincronização Granular

O acesso concorrente a estruturas de dados globais (como as listas de alvos e projéteis) impõe o risco severo de *Race Conditions* e *ConcurrentModificationException*. Para sanar essas vulnerabilidades, foram introduzidos *Locks* (monitores) específicos na classe `Jogo`:

- **lockListas:** Garante exclusividade mútua nas operações de adição, remoção e leitura das coleções de entidades do jogo. Sempre que o otimizador precisa analisar as entidades, ele realiza uma cópia defensiva protegida por este lock.
- **lockColisao:** Um bloqueio adicional focado no motor de física. Quando um projétil atinge as coordenadas de um alvo, este lock previne que duas instâncias tentem destruir o mesmo alvo simultaneamente, garantindo a atomicidade do cálculo da pontuação e energia.

```

public void verificarColisao(Projetoil p){
    synchronized (lockColisao){
        synchronized (lockListas){
            boolean alvoAtingido = false;

            // Verifica na esquerda
            for (int i = 0; i < alvosEsquerda.size(); i++){
                Alvo alvo = alvosEsquerda.get(i);
                double dx = p.getX() - alvo.getX();
                double dy = p.getY() - alvo.getY();
                double distancia = Math.sqrt(dx * dx + dy * dy);

                if (distancia < alvo.getRaio()){
                    alvo.destruir();
                    p.destruir();
                    alvosEsquerda.remove(i);
                    projeteis.remove(p);
                    alvoAtingido = true;
                    break;
                }
            }

            // Se não atingiu na esquerda, tenta na direita
            if (!alvoAtingido) {
                for (int i = 0; i < alvosDireita.size(); i++){
                    Alvo alvo = alvosDireita.get(i);
                    double dx = p.getX() - alvo.getX();
                    double dy = p.getY() - alvo.getY();
                    double distancia = Math.sqrt(dx * dx + dy * dy);

                    if (distancia < alvo.getRaio()){
                        alvo.destruir();
                        p.destruir();
                        alvosDireita.remove(i);
                        projeteis.remove(p);
                        alvoAtingido = true;
                        break;
                    }
                }
            }
        }
    }
}

```

```

if (alvoAtingido) {
    // Pontuação
    if (p.isLadoEsquerdo()){
        pontosEsquerda++;
        energiaEsquerda += 10;
        if (energiaEsquerda > 150) energiaEsquerda = 150;
    } else {
        pontosDireita++;
        energiaDireita += 10;
        if (energiaDireita > 150) energiaDireita = 150;
    }
}
}
}

```

Para evitar a exaustão da CPU através de ciclos ociosos (*busy-waiting*), cada thread implementa descansos estratégicos via `Thread.sleep(16)`, garantindo uma taxa de atualização física cravada em aproximadamente 60 quadros por segundo, libertando ciclos de CPU para o escalonador do Sistema Operacional.

3 API de Afinidade (Thread Affinity) via Reflection

O processamento matemático pesado foi isolado na classe `OtimizadorManager`. Sendo uma operação intensiva de CPU que envolve manipulação matricial contínua, a perda de contexto (*Context Switch*) e a migração de threads entre diferentes núcleos do processador poderiam introduzir latências indesejadas (perda de coerência de cache L1/L2).

Como o framework Android encapsula e restringe as APIs POSIX de baixo nível (como `sched_setaffinity`), a implementação nativa não estava diretamente acessível no espaço de usuário sem bibliotecas JNI em C/C++. Para contornar essa limitação e cumprir o requisito técnico, foi arquitetada uma solução baseada em **Java Reflection**.

O sistema busca dinamicamente a classe oculta `android.os.Process` em tempo de execução e invoca o método `setThreadAffinityMask(int tid, int mask)`. Aplicamos uma máscara de bits (ex: 3, equivalente a 0b11), que instrui o kernel do Linux subjacente a fixar a thread matemática aos dois primeiros núcleos lógicos do processador.

```
private void aplicarThreadAffinityMock() {
    try {
        // Usa Java Reflection para simular a chamada "Process.setThreadAffinityMask" pedida na AV2
        Class<?> processClass = Class.forName( className: "android.os.Process");
        Method setAffinityMethod = processClass.getMethod( name: "setThreadAffinityMask", int.class, int.class);
        int myTid = (int) processClass.getMethod( name: "myTid").invoke( o: null);

        // Supondo que quer rodar nos 2 primeiros cores (mask = 3 -> binário 11)
        setAffinityMethod.invoke( o: null, ...objects: myTid, 3);
        System.out.println("Thread affinity setada com sucesso (Simulação)");
    } catch (Exception e) {
        // Em caso do OS não suportar essa API internamente (o que é o comum para apps de usuário sem root)
        System.out.println("Dummy Thread Affinity aplicado. Fallback para execução normal.");
    }
}
```

Figura 2: Invocação dinâmica da API de Afinidade do kernel do Android através de Reflection.

4 Modelagem Estocástica e Reconciliação de Dados

Em sistemas de automação real, os dados aferidos do ambiente não são perfeitamente precisos. Para simular esta degradação informacional, o `AutoTarget` implementa um modelo de ruído e seu consequente filtro corretivo.

4.1 Sensores Virtuais Ruidosos

O posicionamento exato de cada Alvo é mascarado pela classe `SensorVirtual`, que aplica variações pseudo-aleatórias nas coordenadas X e Y. Cada alvo armazena um *buffer* tem-

poral de suas próprias leituras recentes. A tomada de decisão dos canhões nunca acessa a posição verdadeira do alvo, mas sim este histórico ruidoso.

4.2 Reconciliação Matemática de Medições

Para extrair o valor verdadeiro a partir de múltiplos dados imperfeitos, emprega-se o método matricial de Reconciliação de Dados, suportado pela biblioteca algébrica **Apache Commons Math3**.

O algoritmo resolve a minimização de mínimos quadrados utilizando a matriz de variância (V), a matriz de incidência espacial (A) e o vetor de medições brutas (y). A equação fundamental aplicada para obter as medições reconciliadas ($\hat{y}$) é expressa por:

$$\hat{y} = y - VA^T(AVA^T)^{-1}Ay \quad (1)$$

A inversão do componente (AVA^T) é computacionalmente custosa e sensível a matrizes singulares, razão pela qual utilizamos a **Decomposição LU** (**LUDecomposition**). O resultado é um conjunto de distâncias purificadas, permitindo que a inteligência artificial faça estimativas acuradas da topologia da arena.

```

public class DataReconciliation {

    /**
     * Aplica a reconciliação de dados na medição de distâncias.
     * Formula:  $y_{\hat{}} = y - V * A^T * (A * V * A^T)^{-1} * A * y$ 
     */
    1 usage
    public static double[] reconcile(double[] y_arr, double[][] V_arr, double[][] A_arr) {
        try {
            RealVector y = MatrixUtils.createRealVector(y_arr);
            RealMatrix V = MatrixUtils.createRealMatrix(V_arr);
            RealMatrix A = MatrixUtils.createRealMatrix(A_arr);

            RealMatrix AT = A.transpose();
            RealMatrix AVAT = A.multiply(V).multiply(AT);

            // Inversão da matriz usando decomposição LU
            RealMatrix invAVAT = new LUDecomposition(AVAT).getSolver().getInverse();

            // Calculando a correção:  $V * A^T * (A * V * A^T)^{-1} * A * y$ 
            RealVector correction = V.multiply(AT).multiply(invAVAT).multiply(A).operate(y);

            //  $y_{\hat{}} = y - correction$ 
            RealVector y_hat = y.subtract(correction);

            return y_hat.toArray();
        } catch (Exception e) {
            // Em caso de matrizes singulares ou erros dimensionais (ruído zero, etc)
            e.printStackTrace();
            return y_arr; // fallback para as medidas originais
        }
    }
}

```

Figura 3: Implementação do solver matricial utilizando Decomposição LU para a Reconciliação de Dados.

5 Modelagem Comportamental: Agentes Racionais e Autonomia

Além da resolução mecânica, os canhões no AutoTarget operam sob o paradigma de Agentes Racionais de Inteligência Artificial, tomando decisões baseadas no estado do mundo sob forte restrição de recursos.

5.1 Cinemática Horizontal e Prevenção de Sobreposição

A cada 500 milissegundos, o `OtimizadorManager` dita novos *setpoints* aos canhões. O algoritmo iterativo localiza o alvo estocasticamente mais próximo e traça o vetor de per-

seguição horizontal.

Para impedir que a lógica heurística empilhe todos os canhões na mesma coordenada (o que violaria os princípios de física básica da simulação), foi introduzido um algoritmo de "Repulsão Progressiva". O motor ordena as posições matemáticas ideais e aplica um espaçamento radial forçado (constante de 40 pixels). Se dois canhões tentam convergir para o mesmo alvo, o sistema os espalha de maneira ótima em torno do ponto de interesse, mantendo as restrições limítrofes do cenário.

5.2 Função de Utilidade e Alocação Dinâmica

Manter artilharia em campo possui um custo de energia ativo por segundo, que só é superado pelo ganho obtido no abate de alvos. A economia global dos times é gerida autonomamente por uma **Função de Utilidade** baseada em Retorno sobre Investimento (ROI).

A equação de utilidade penaliza quantitativos de canhões superiores a 5 (através de um multiplicador de custo) e bonifica a densidade de alvos num raio local inferior a 300 pixels. Se a predição resulta em perda líquida de energia esperada, a IA desativa preventivamente a própria artilharia, encolhendo suas fileiras. Se as reservas excedem limites de segurança e múltiplos alvos adentram a zona de predição, a IA instancia novos canhões de forma dinâmica, realizando expansão agressiva.

```
private void avaliarUtilidadeCanhoes(boolean isEsquerda, int numCanhoes, int numAlvos, double[] distanciasReconciliadas) {
    // Função de utilidade: Cada alvo próximo (< 300) gera "pontos esperados"
    int abatesEsperados = 0;
    for (double d : distanciasReconciliadas) {
        if (d < 300) abatesEsperados++;
    }

    double ganhoPorAbate = 10.0;
    double custoManutencao = numCanhoes * 1.0; // 1 de energia/s = 10 em 10s
    double fatorPenalidade = numCanhoes > 5 ? (1 + (numCanhoes - 5) * 0.2) : 1.0;

    double utilidadeAtual = (abatesEsperados * ganhoPorAbate) - (custoManutencao * fatorPenalidade);

    // Se a utilidade estiver muito baixa e o time tiver mais de 1 canhão, compensa remover
    if (utilidadeAtual < 0 && numCanhoes > 1) {
        jogo.removerCanhao(isEsquerda);
    }

    // Se houver muitos alvos para os canhões atuais (e energia suficiente), adicionamos
    else if (abatesEsperados > numCanhoes * 2 && numCanhoes < 10) {
        int energia = isEsquerda ? jogo.getEnergiaEsquerda() : jogo.getEnergiaDireita();
        if (energia > 50) { // Margem de segurança
            try {
                double novoX = isEsquerda ? (jogo.getLarguraTela() * 0.2) : (jogo.getLarguraTela() * 0.8);
                double novoY = jogo.getAlturaTela() - 50;
                jogo.adicionarCanhao(novoX, novoY);
            } catch (JogoException e) {
                // Ignora, limite máximo atingido
            }
        }
    }
}
```

Figura 4: Lógica heurística de Função de Utilidade para alocação autônoma de unidades de combate.

6 Pipeline de Renderização Não-Bloqueante

Garantir que a complexidade computacional descrita não intervisse na fluidez da interação com o usuário foi um desafio considerável. O paradigma tradicional da classe `View` no Android roda de forma síncrona com a manipulação de eventos do usuário, o que geraria *frame drops* (engasgos) visíveis e travamentos.

A arquitetura foi subvertida utilizando um `SurfaceView`, que reserva um buffer de memória direto na placa de vídeo (*hardware frame buffer*). Uma thread dedicada unicamente à renderização visual (`threadDesenho`) aplica *locks* de curto ciclo na *Canvas* gráfica, injeta as coordenadas das entidades processadas pelas demais threads, e dispara o comando de *post* para o *display*, resultando em uma taxa de atualização impecável e permitindo escalabilidade para centenas de objetos virtuais simultâneos.

```
Canvas canvas = null;
try {
    // 1. "Tranca" a tela para começar a desenhar
    canvas = prancheta.lockCanvas();

    } catch (Exception e){
        e.printStackTrace();
    } finally {
        if (canvas != null){
            // 7. "Destranca" a tela e mostra o desenho final para o jogador
            prancheta.unlockCanvasAndPost(canvas);
        }
    }

    // Limita o loop a ~60 FPS para não consumir CPU desnecessariamente
    try { Thread.sleep( millis: 16); } catch (InterruptedException ignored) {}
```

Figura 5: Ciclo de renderização assíncrona desacoplado da UI Thread principal.

7 Conclusão

O sistema **AutoTarget** consolida uma arquitetura de simulação paralela de alta complexidade. Foram solucionadas as intempéries inerentes à concorrência multi-núcleo em Java através de modelagem de locks rígidos. A introdução de simulação de ruídos sensoriais com seu respectivo saneamento matricial (Decomposição LU), bem como o autogerenciamento de inteligência artificial sobre os times, culminou em uma aplicação engenhosa, escalável e tecnicamente profunda. Todo este peso de processamento opera nos bastidores de um motor gráfico construído sob medida para ser resiliente a travamentos, atingindo em sua plenitude as diretrizes propostas na segunda avaliação da disciplina.

Referências

- [1] Google. *Android Developers Documentation: SurfaceView and Canvas*. Disponível em: <https://developer.android.com/reference/android/view/SurfaceView>. Acesso em: 25 maio 2026.
- [2] The Apache Software Foundation. *Apache Commons Math 3.6 API: LUDecomposition*. Disponível em: <https://commons.apache.org/proper/commons-math/javadocs/api-3.6/>. Acesso em: 25 maio 2026.
- [3] Oracle. *Java Concurrency and Multithreading Tutorial*. Disponível em: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>. Acesso em: 25 maio 2026.
- [4] Romagnoli, J. A., & Sánchez, M. C. (2000). *Data Processing and Reconciliation for Chemical Process Operations*. Academic Press.
- [5] Linux Programmer's Manual. *sched_setaffinity(2) - Linux man page*. Disponível em: https://man7.org/linux/man-pages/man2/sched_setaffinity.2.html. Acesso em: 25 maio 2026.